Integrated Technical Design Document (TDD)

TSI Ledger Service: Enterprise Double-Entry Subsystem modeled on TSI DPDP CMS

1. System Architecture & Component Topology

The TSI Ledger Service is a production-grade, highly secure double-entry financial accounting subsystem. Modeled structurally on the container-first patterns of the **TSI DPDP CMS framework**, it decouples computational ledger invariants from the persistent storage engine.

The application layers run containerized over an isolated bridge network environment. The functional core uses Java 17 deployed inside an Eclipse Temurin/Jetty web container engine interacting with a dedicated PostgreSQL database cluster instance.

Core Architecture Guarantees

- **Strict Double-Entry Matching:** Subsystem verification mandates that for every transaction record, the cumulative total of credit legs balances to the zero margin against the cumulative debit legs.

$$\sum Debits - \sum Credits = 0$$

- **Cryptographic Chain Immutability:** Conforming to BSA reporting profiles, entries are structurally anchored back-to-front using a persistent SHA-256 hashchain pipeline.
- **Tenant Environment Isolation:** Application spaces are managed dynamically using isolated scope identifiers validated by a low-level authentication mapping layer.

2. Environment Configuration Matrix (`.env`)

Secrets, local target environments, connection contexts, and processing thresholds are distributed out-of-band using standard environment templates matching the deployment design criteria of core TSI microservices.

```
# ================================================================================
# TSI Ledger Engine Configuration Variables
# ================================================================================
APP_ENV=production
SERVER_PORT=8080

# --- Database Persistent Connectivity Settings ---
DB_HOST=postgres_db
DB_PORT=5432
DB_NAME=tsi_ledger_prod
DB_USER=ledger_fiduciary_admin
DB_PASSWORD=SecureMasterPasswordStringEx2026

# --- Security and Cryptographic Salts ---
LEDGER_HMAC_SALT=d3b07384d113edec49eaa6238ad5ff00b7d9eb21bbb7109b6929

# --- Microservice Engine Capacities ---
```

```
MAX_DB_POOL_SIZE=25
IDEMPOTENCY_CACHE_TTL_MINUTES=60
```

3. Persistent Schema Engineering (`init.sql`)

Database layout choices enforce data types over broad text models to ensure strict computational exactness. Float configurations are explicitly banned to prevent rounding leakage across balancing calculations.

```
-- 1. Fiduciary Applications Tenant Container
CREATE TABLE fiduciary_apps (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_name VARCHAR(255) NOT NULL,
    api_key_hash VARCHAR(64) NOT NULL UNIQUE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_apps_hash ON fiduciary_apps(api_key_hash);

-- 2. Chart of Accounts Table Layout
CREATE TABLE accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    app_id UUID NOT NULL REFERENCES fiduciary_apps(id) ON DELETE RESTRICT,
    account_code VARCHAR(32) NOT NULL,
    account_name VARCHAR(128) NOT NULL,
    account_type VARCHAR(16) NOT NULL CHECK (account_type IN ('ASSET', 'LIABILITY', 'EQUITY',
'REVENUE', 'EXPENSE')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_app_account_code UNIQUE (app_id, account_code)
);

-- 3. Core Journal Block Entry Record Ledger
CREATE TABLE journal_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    app_id UUID NOT NULL REFERENCES fiduciary_apps(id) ON DELETE RESTRICT,
    sequence_number BIGINT NOT NULL,
    reference_id VARCHAR(128) NOT NULL,
    description TEXT NOT NULL,
    posted_at TIMESTAMP WITH TIME ZONE NOT NULL,
    entry_payload_hash CHARACTER(64) NOT NULL,
    previous_block_hash CHARACTER(64) NOT NULL,
    block_hash CHARACTER(64) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_app_sequence UNIQUE (app_id, sequence_number)
);
CREATE INDEX idx_entries_audit_chain ON journal_entries(app_id, sequence_number DESC);

-- 4. Entry Transaction Legs Table
CREATE TABLE entry_lines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entry_id UUID NOT NULL REFERENCES journal_entries(id) ON DELETE RESTRICT,
    account_id UUID NOT NULL REFERENCES accounts(id) ON DELETE RESTRICT,
    line_type VARCHAR(6) NOT NULL CHECK (line_type IN ('DEBIT', 'CREDIT')),
    amount NUMERIC(18, 4) NOT NULL CHECK (amount > 0),
```

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

4. Core API Engine Interface Specification

All incoming interface connections must deliver authorization signatures through the `X-API-Key` tracking header. If authorization maps fail database matching indices, the engine triggers an early `401 Unauthorized` block.

4.1 Entry Submission Endpoint: `POST /v1/entries`

Submits accounting data payload items into the execution pipeline context. Content logic confirms balanced inputs inside standard transaction guarantees before running cryptographic serialization loops.

```
-- Sample Payload Input Shape
{
  "reference_id": "TXN-2026-8891",
  "description": "Subscription Revenue Processing",
  "posted_at": "2026-07-06T06:12:00Z",
  "lines": [
    { "account_code": "1010", "type": "DEBIT", "amount": 1250.00 },
    { "account_code": "4000", "type": "CREDIT", "amount": 1250.00 }
  ]
}
```

4.2 Audit Verification Verification Endpoint: `GET /v1/audit/verify`

Executes structural audits down the linear transaction sequences mapped to the application tenant. The process tests internal cryptographic values row by row to identify anomalies out-of-band.

4.3 Reporting Endpoints: `GET /v1/reports/balance-sheet`

Computes financial summary values directly from the immutable transaction log records. Asset tracking aggregates debits over credits while tracking parameters shift inverse along liabilities and equity structures to satisfy the ledger balance equation:

$$Assets = Liabilities + Equity$$

5. Cryptographic Hashing Engine Pipeline Logic

The transactional execution scope applies deterministic serialization logic against the tracking values. As new blocks register, the engine increments a gapless, application-locked sequence integer value (n). The structural signature calculations are defined as follows:

$$P_n = SHA256(CanonicalJSON(DataPayload_n))$$

$$H_n = SHA256(P_n + H_{n-1})$$

If structural changes occur below the existing sequence blocks via unauthenticated query tools, validation procedures break down the sequence indexes, identifying the breach point.

6. Application Orchestration Manifest (`docker-compose.yml`)

```
version: '3.8'
services:
  postgres_db:
    image: postgres:15-alpine
    container_name: tsi_ledger_postgres
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - ledger_data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    ports:
      - "5432:5432"
    networks:
      - ledger_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${DB_USER} -d $${DB_NAME}"]
      interval: 5s
      timeout: 5s
      retries: 5

  jetty_app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: tsi_ledger_engine
    ports:
      - "${SERVER_PORT}:8080"
    env_file:
      - .env
    depends_on:
      postgres_db:
        condition: service_healthy
    networks:
      - ledger_network
    restart: always

networks:
  ledger_network:
    driver: bridge

volumes:
  ledger_data:
    driver: local
```